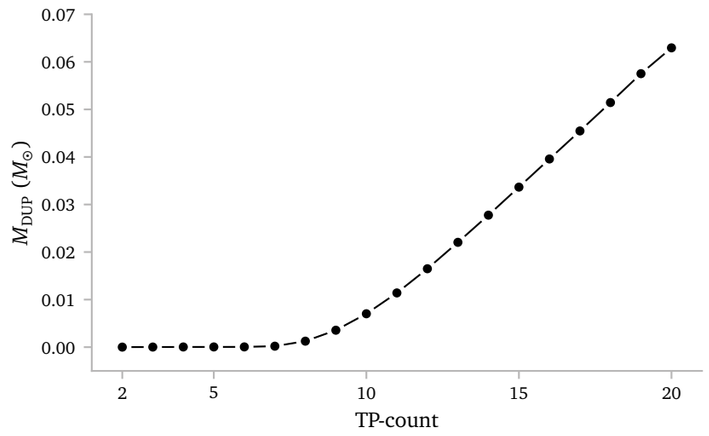
Koen Essers • Master Internship • July 11, 2026

Notes Week 22

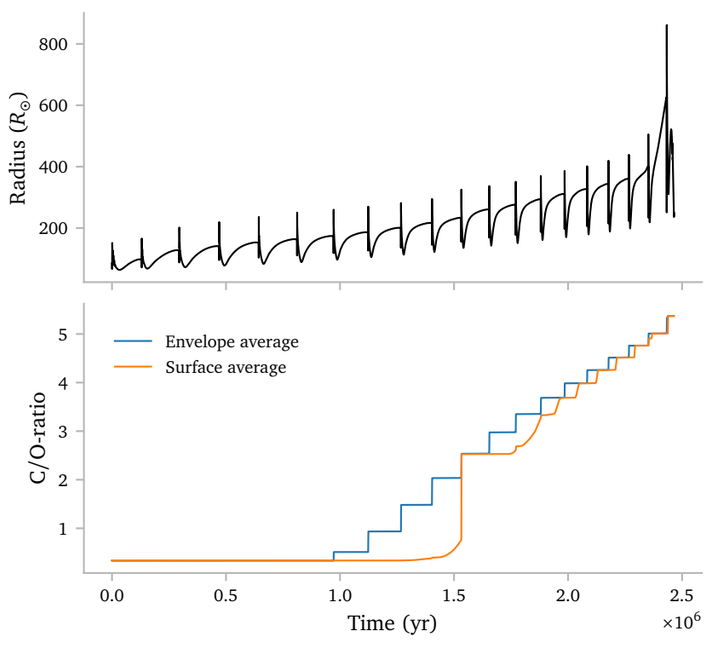
a. Dredge-Up mass

Since the current plan is to use the amount of mass that has been dredged up and the intershell abundances to infer the envelope abundances, I must be sure that I can actually accurately get this dredged up mass.

Below I show that this is indeed possible, and the values align roughly with what I would expect from Figure 1 of Karakas & Lugaro (2016).



b. Envelope Abundances



c. Dwarf Metallicity

I am using $[\mathrm{Fe/H}] = -0.4$ which equates to $z \approx 0.00557$.

d. Extended Grid

I want to simulate TPAGB stars from 1 to 3.5 $M_\odot$ in order to make sure I get the minimum mass for TPAGB stars.

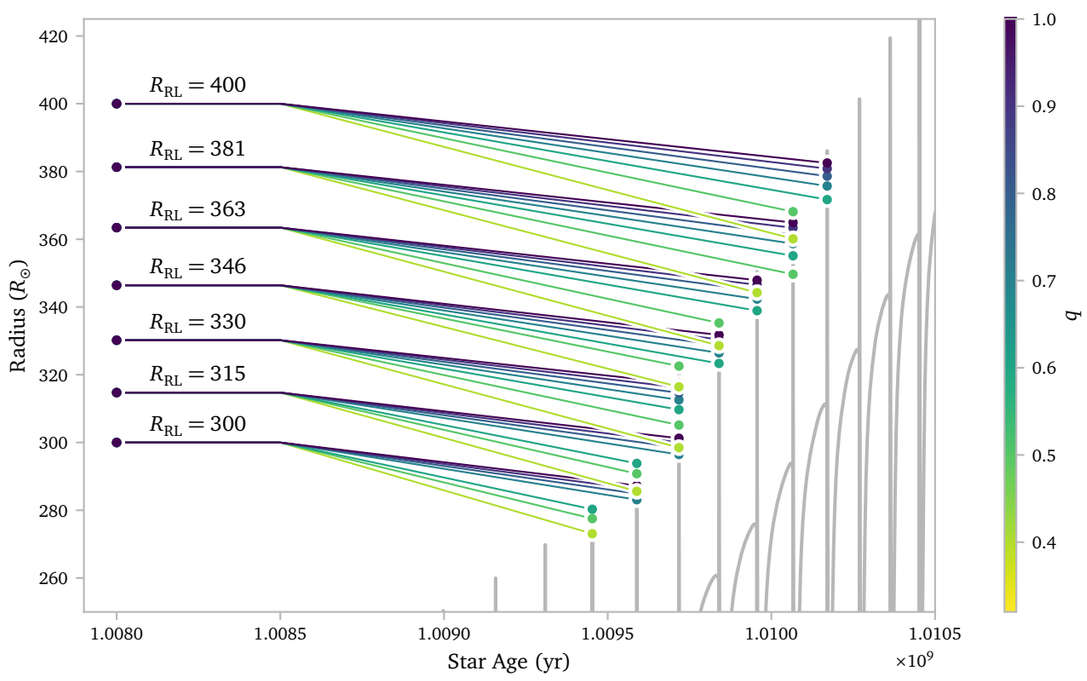
Initially, I use a dense spacing of $\Delta M = 0.1 M_\odot$. This is probably a bit much to construct grids with, but at least I can clearly find the starting mass of the TPAGB, and I can see the stability of a wide range of masses.

e. Optimizing Output

Since we will be dealing with much larger grids and more masses, I think it is very much time to optimize the grid generation code. Up to now, it would read in the compute star model to use as input. This is very slow.

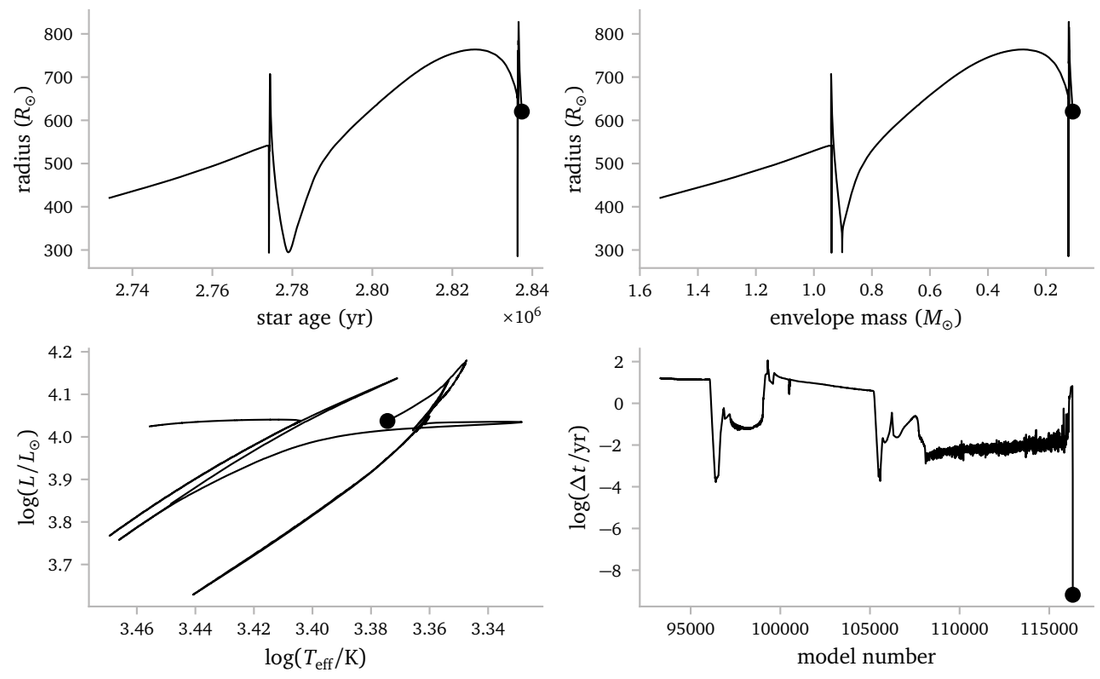
My idea is to create some optimized binary file that contains only the necessary data for the grid generation.

f. Spacing



g. Debugging!

The 2.4 $M_\odot$ star was behaving *REALLY* badly, and would not at all continue its evolution, despite all sorts of attempts to help the solver. Because of this, I decided to look into *what* exactly is going wrong. Let me start by plotting where the problem starts occuring.



It seems like the star cannot continue evolving just after its last thermal pulse. The radius and HR diagram look fine, but we see that the timestep goes to 0.

The terminal output shows me the usual ÆSOPUS warning, as well as the eos warning. I need to dig a bit deeper to figure out what is actually going wrong.

To start, I created a model just before the point where the star crashes. I also use the general debug options available to me in the inlists.

To start, I see this:

```
               solver_call_number          1
             gold tolerances level         1
   correction tolerances: norm, max    3.0000000000000001D-05    3.0000000000000001D-03
tol1 residual tolerances: norm, max    9.9999999999999994D-12    1.0000000000000001D-09
1  1  coeff 1.0000 avg resid  0.691E-05     max resid dv_dt     1  0.83380E+00 mix type xx000
                  avg corr  0.206E-03      max corr lnd  1986  0.53235E-02 mix type 00000
                  avg+max corr+resid
1  2  coeff 1.0000 avg resid  0.683E-05     max resid dv_dt     1  0.82426E+00 mix type xx000
                  avg corr  0.573E-04      max corr lnd     2  0.12612E-01 mix type x0000
                  avg+max corr+resid
do_eos_for_cell: get_eos ierr          -1
do_eos_for_cell: get_eos ierr          -1
do_eos_for_cell: get_eos ierr          -1
do_eos_for_cell: get_eos ierr          -1
do_eos_for_cell: get_eos ierr          -1
do_eos_for_cell: get_eos ierr          -1
do_eos_for_cell: get_eos ierr          -1
do_eos_for_cell: get_eos ierr          -1
do_eos_for_cell: get_eos ierr          -1
do_eos_for_cell: get_eos ierr          -1
do_eos returned ierr          -1
set_hydro_vars failed in call to set_micro_vars
                  set_hydro_vars returned ierr          -1
  eval_equations: set_solver_vars returned ierr          -1
adjust_correction: eval_equations returned ierr          -1    1.0D+00    1.0D+00
```

The first error occurs in the `do_eos_for_cell` call!

I then use the GNU Project Debugger `gdb` to debug *MESA*. I write:

```
> OMP_NUM_THREADS=1 gdb ./star
(gdb) b eos_support.f90:89
(gdb) condition 1 logRho .lt. -25
```

The first line activates the debugger with the `star` executable where `OMP_NUM_THREADS=1` makes sure we are not multithreading. The second line creates a breakpoint at line 89 from the file `eos_support.f90`. Line 89 is the condition in `get_eos` that returns the error -1. The third line tell the breakpoint to only break if `logRho` < -25, which is the exact condition that returns the error.

This is the (relevant) output.

```
               solver_call_number          1
             gold tolerances level         1
   correction tolerances: norm, max    3.0000000000000001D-05    3.0000000000000001D-03
tol1 residual tolerances: norm, max    9.9999999999999994D-12    1.0000000000000001D-09
1  1  coeff 1.0000 avg resid  0.691E-05     max resid dv_dt     1  0.83380E+00 mix type xx000
                  avg corr  0.206E-03      max corr lnd  1986  0.53235E-02 mix type 00000
                  avg+max corr+resid
1  2  coeff 1.0000 avg resid  0.683E-05     max resid dv_dt     1  0.82426E+00 mix type xx000
                  avg corr  0.573E-04      max corr lnd     2  0.12612E-01 mix type x0000
                  avg+max corr+resid

Breakpoint 1, eos_support::get_eos (s=0xe7c0e0 <__star_data_def_MOD_star_handles>, k=1,
                                    xa=..., rho=<optimized out>, logrho=-40.561141182088775,
                                    t=228.31745818308877, logt=2.3585391208756734,
                                    res=..., dres_dlnrho=..., dres_dlnt=..., dres_dxa=...,
                                    ierr=0) at ../private/eos_support.f90:89
89            if(logRho < -25) then
```

This is the relevant source code:

```
59  ! $MESA_DIR/star/private/eos_support.f90
60
61  subroutine get_eos( &
62             s, k, xa, &
63             Rho, logRho, T, logT, &
64             res, dres_dlnRho, dres_dlnT, &
65             dres_dxa, ierr)
66
67          use eos_lib, only: eosDT_get
68          use eos_def, only: num_eos_basic_results,
            ↪  num_eos_d_dxa_results, num_helm_results,
            ↪  i_lnE
69
70          type (star_info), pointer :: s
71          integer, intent(in) :: k ! 0 means not being
            ↪  called for a particular cell
72          real(dp), intent(in) :: xa(:), Rho, logRho, T,
            ↪  logT
73          real(dp), dimension(num_eos_basic_results),
            ↪  intent(out) :: &
74             res, dres_dlnRho, dres_dlnT
75          real(dp), intent(out) ::
            ↪  dres_dxa(num_eos_d_dxa_results,s% species)
76          integer, intent(out) :: ierr
77
78          real(dp), dimension(num_eos_basic_results) ::
            ↪  dres_dabar, dres_dzbar
79          integer :: j
80          logical :: off_table
81
82          include 'formats'
83
84          ierr = 0
85
86          if (s% doing_timing) &
87              s% timing_num_get_eos_calls = s%
                ↪  timing_num_get_eos_calls + 1
88
89          if(logRho < -25) then
90              ! Provide some hard lower limit on what we
                ↪  would even try to evalue the eos at
91              ! Going to low causes FPE's when we try to
                ↪  evaluate certain derviatives that need
                ↪  (rho**power)
92              s% retry_message = 'eos evaluted at too low a
                ↪  density'
93              ierr = -1
94              return
95          end if
96      ...
```

Clearly, something is going wrong with the *density*! In this case, the first zone (k=1, at the surface) has a density of $\log(\rho) \approx -40$. Obviously this is insane. The temperature is very low as well and wrong.

So, really we need to look back a little bit, not at `get_eos` where the *error* originates, but a little before that, where the back values originate.

Below, I show the source code that calls `get_eos`:

```
! $MESA_DIR/star/private/micro.f90

subroutine do_eos_for_cell(s,k,ierr)

    use const_def
    use chem_def
    use chem_lib
    use eos_def
    use eos_lib, only: eval_PC_fraction
    use eos_support
    use net_def,only:net_general_info
    use star_utils, only: write_eos_call_info

    type (star_info), pointer :: s
    integer, intent(in) :: k
    integer, intent(out) :: ierr

    real(dp), dimension(num_eos_basic_results) :: &
         res, res_a, res_b, d_dlnd, d_dlnT, d_eos_dabar, d_eos_dzbar
    real(dp) :: &
         sumx, dx, dxh_a, dxh_b, &
         Rho, logRho, lnd, lnE, logT, T, energy, logQ, frac
    integer, pointer :: net_iso(:)
    integer :: j, species
    real(dp), parameter :: epsder = 1d-4, Z_limit = 0.5d0

    logical, parameter :: testing = .false.

    include 'formats'

    ierr = 0

    net_iso => s% net_iso
    species = s% species
    call basic_composition_info( &
         species, s% chem_id, s% xa(1:species,k), s% X(k), s% Y(k), s% Z(k), &
         s% abar(k), s% zbar(k), s% z2bar(k), s% ye(k), s% mass_correction(k), sumx)

    logT = s% lnT(k)/ln10

    if (s% lnPgas_flag) then
       if (s% Pgas(k) == 0) then
          ierr = -1
          return
          write(*,2) 's% lnPgas(k)/ln10', k, s% lnPgas(k)/ln10
          stop 'do_eos_for_cell s% Pgas(k) == 0'
       end if
       call get_peos( &
            s, k, s% Z(k), s% X(k), s% abar(k), s% zbar(k), s% xa(:,k), &
            s% Pgas(k), s% lnPgas(k)/ln10, s% T(k), logT, &
            Rho, logRho, s% dlnRho_dlnPgas_const_T(k), s% dlnRho_dlnT_const_Pgas(k), &
            res, s% d_eos_dlnd(:,k), s% d_eos_dlnT(:,k), &
            d_eos_dabar, d_eos_dzbar, ierr)
       s% lnd(k) = logRho*ln10
       s% rho(k) = Rho
       call store_stuff(ierr)
       return
    end if

    logRho = s% lnd(k)/ln10

    call get_eos( &
         s, k, s% Z(k), s% X(k), s% abar(k), s% zbar(k), s% xa(:,k), &
         s% rho(k), logRho, s% T(k), logT, &
         res, s% d_eos_dlnd(:,k), s% d_eos_dlnT(:,k), &
         d_eos_dabar, d_eos_dzbar, &
         ierr)
    if (ierr /= 0) then
       if (s% report_ierr) then
          write(*,*) 'do_eos_for_cell: get_eos ierr', ierr
          stop 'do_eos_for_cell'
       end if
       return
    end if
    s% lnPgas(k) = res(i_lnPgas)
    s% Pgas(k) = exp_cr(s% lnPgas(k))
    ...
```

The errors occur in line 344. We can also look at the backtrace of the get_eos call:

```
(gdb) bt
#0  eos_support::get_eos (s=0xe7c0e0
 ↪  <__star_data_def_MOD_star_handles>, k=1, xa=...,
 ↪  rho=<optimized out>, logrho=-40.561141182088775,
 ↪  t=228.31745818308877, logt=2.3585391208756734,
    res=..., dres_dlnrho=..., dres_dlnt=...,
     ↪  dres_dxa=..., ierr=0) at
     ↪  ../private/eos_support.f90:89
#1  0x0000000000503d11 in micro::do_eos_for_cell
 ↪  (s=<optimized out>, k=<optimized out>, ierr=0) at
 ↪  ../private/micro.f90:385
#2  0x00000000004422ec in
 ↪  star_utils::__star_utils_MOD_foreach_cell._omp_fn.0
 ↪  () at ../private/star_utils.f90:72
#3  0x00007fea9e211f42 in GOMP_parallel (fn=0x442260
 ↪  <star_utils::__star_utils_MOD_foreach_cell._omp_fn.0>,
 ↪  data=0x7ffcbdc100b0, num_threads=1, flags=0)
    at
     ↪  /home/user/sdk2-tmp/build/gcc/libgomp/parallel.c:171
#4  0x0000000000456935 in star_utils::foreach_cell
 ↪  (s=0xe7c0e0 <__star_data_def_MOD_star_handles>,
 ↪  nzlo=<optimized out>, nzhi=<optimized out>,
 ↪  use_omp=<optimized out>,
    do1=0x5036c0 <micro::do_eos_for_cell>, ierr=0) at
     ↪  ../private/star_utils.f90:72
#5  0x00000000005057c3 in micro::do_eos (ierr=0,
 ↪  nzhi=<optimized out>, nzlo=<optimized out>,
 ↪  s=<optimized out>) at ../private/micro.f90:271
#6  micro::set_eos (ierr=0) at ../private/micro.f90:217
#7  micro::set_micro_vars (nzlo=1, nzhi=5857,
 ↪  skip_eos=.FALSE., skip_net=.FALSE., skip_neu=.FALSE.,
 ↪  skip_kap=.FALSE., ierr=0) at ../private/micro.f90:97
#8  0x000000000050cbe7 in hydro_vars::set_hydro_vars
 ↪  (s=0xe7c0e0 <__star_data_def_MOD_star_handles>,
 ↪  nzlo=1, nzhi=5857, skip_basic_vars=.TRUE.,
 ↪  skip_micro_vars=.FALSE.,
    skip_m_grav_and_grav=.TRUE., skip_eos=.FALSE.,
     ↪  skip_net=.FALSE., skip_neu=.FALSE.,
     ↪  skip_kap=.FALSE., skip_grads=.TRUE.,
     ↪  skip_rotation=.TRUE., skip_brunt=.TRUE.,
    skip_other_cgrav=.TRUE., skip_mixing_info=.TRUE.,
     ↪  skip_set_cz_bdy_mass=.TRUE., skip_mlt=.FALSE.,
     ↪  ierr=0) at ../private/hydro_vars.f90:506
#9  0x00000000007fa3fd in
 ↪  solver_support::set_vars_for_solver (s=0xe7c0e0
 ↪  <__star_data_def_MOD_star_handles>, nvar=13, nzlo=1,
 ↪  nzhi=<optimized out>, dt=69894018.156770468, ierr=0)
    at ../private/solver_support.f90:1107
#10 0x00000000007fe7a9 in solver_support::set_solver_vars
 ↪  (ierr=0, dt=69894018.156770468, nvar=13, s=0xe7c0e0
 ↪  <__star_data_def_MOD_star_handles>) at
 ↪  ../private/solver_support.f90:815
#11 solver_support::eval_equations (s=0xe7c0e0
 ↪  <__star_data_def_MOD_star_handles>, nvar=13, ierr=0)
 ↪  at ../private/solver_support.f90:113
#12 0x00000000007bd036 in do_solver::do_equations
 ↪  (ierr=0) at ../private/star_solver.f90:736
#13 0x0000000000c2960 in star_solver::adjust_correction
 ↪  (_err_msg=256, _err_msg=256, ierr=0, err_msg=<error
 ↪  reading variable: Cannot access memory at address
 ↪  0x9>, coeff=1, slope=0, f=0,
    grad_f=<error reading variable: value requires 609128
     ↪  bytes, which is more than max-value-size>,
     ↪  max_corr_coeff=1, min_corr_coeff_in=<optimized
     ↪  out>) at ../private/star_solver.f90:848
#14 star_solver::do_solver (s=0xe7c0e0
 ↪  <__star_data_def_MOD_star_handles>, nvar=13, af1=...,
 ↪  ldaf=39, neq=76141,
 ↪  skip_global_corr_coeff_limit=.TRUE.,
 ↪  gold_tolerances_level=1,
    tol_max_correction=0.003000000000000001,
     ↪  tol_correction_norm=3.0000000000000001e-05,
     ↪  work=<error reading variable: value requires
     ↪  61521928 bytes, which is more than
     ↪  max-value-size>,
    lwork=7690241, iwork=<error reading variable: value
     ↪  requires 304564 bytes, which is more than
     ↪  max-value-size>, liwork=76141,
     ↪  convergence_failure=.FALSE., ierr=0)
    at ../private/star_solver.f90:317
#15 0x00000000007c5823 in star_solver::solver (s=0xe7c0e0
 ↪  <__star_data_def_MOD_star_handles>, nvar=13,
 ↪  skip_global_corr_coeff_limit=.TRUE.,
 ↪  gold_tolerances_level=1,
    tol_max_correction=0.003000000000000001,
     ↪  tol_correction_norm=3.0000000000000001e-05,
     ↪  work=<error reading variable: value requires
     ↪  61521928 bytes, which is more than
     ↪  max-value-size>,
    lwork=7690241, iwork=<error reading variable: value
     ↪  requires 304564 bytes, which is more than
     ↪  max-value-size>, liwork=76141,
     ↪  convergence_failure=.FALSE., ierr=0)
    at ../private/star_solver.f90:104
```

I think the interesting calls are #8 and #13.

- Function call 8 sets the hydro_vars, which later give the error.
- Function call 13 adjusts the corrections after the previous newton iteration was unable to converge.

To continue debugging, I need to figure out *where* the wrong density values come from.

To do this, I use grep to look for all occurrences of density assignments:

```
> grep -R "s% *lnd.*=" star/private
star/private/predictive_mix.f90:            s% lnd(k) = lnd_save(k)
star/private/hydro_rsp2_support.f90:        s% lnd(k) = log(s% rho(k))
star/private/micro.f90:                     s% lnd(k) = logRho*ln10
star/private/star_utils.f90:                s% lnd_start(k) = -1d99
star/private/rsp_eval_eos_and_kap.f90:      s% lnd(k) = logRho*ln10
star/private/rsp_eval_eos_and_kap.f90:      s% lnd(k) = logRho*ln10
star/private/rsp_eval_eos_and_kap.f90:      s% lnd(kk) = logRho_result*ln10
star/private/conv_premix.f90:               s%lnd(kc_t:kc_b) = sd%lnd(kc_t:kc_b)
star/private/hydro_vars.f90:                s% lnd(k) = s% xh(i_lnd,k)
star/private/hydro_vars.f90:                s% lnd_start(k) = s% lnd(k)
star/private/solver_support.f90:            s% lnd(k) = x(i_lnd)
star/private/struct_burn_mix.f90:           !s% lnd_start(k) = s% lnd(k) set elsewhere
star/private/write_model.f90:               call write1(s% lnd(k),ierr); if (ierr /= 0) exit
```

Really, there are only a two lines here, namely:

```
star/private/hydro_vars.f90:                s% lnd(k) = s% xh(i_lnd,k)
star/private/solver_support.f90:            s% lnd(k) = x(i_lnd)
```

We can again run the debugger and set breakpoints at these lines:

```
(gdb) b hydro_vars.f90:299
Breakpoint 1 at 0x50dbb2: file ../private/hydro_vars.f90, line 299.
(gdb) b solver_support.f90:1337
Breakpoint 2 at 0x7f7194: file ../private/solver_support.f90, line 1337.
(gdb) condition 2 x(i_lnd) < -30
(gdb) run
```

Eventually, we reach the state:

```
               solver_call_number           1
             gold tolerances level          1
   correction tolerances: norm, max    3.0000000000000001D-05    3.0000000000000001D-03
tol1 residual tolerances: norm, max    9.9999999999999994D-12    1.0000000000000001D-09
1  1  coeff 1.0000 avg resid  0.691E-05     max resid dv_dt     1  0.83380E+00 mix type xx000
                   avg corr  0.206E-03      max corr lnd  1986  0.53235E-02 mix type 00000
                   avg+max corr+resid
1  2  coeff 1.0000 avg resid  0.683E-05     max resid dv_dt     1  0.82426E+00 mix type xx000
                   avg corr  0.573E-04      max corr lnd     2  0.12612E-01 mix type x0000
                   avg+max corr+resid

Breakpoint 2, set_vars_for_solver::set1 (k=1, report=.FALSE., ierr=0)
             at ../private/solver_support.f90:1337
1337                        s% lnd(k) = x(i_lnd)
(gdb) p x(i_lnd)
$1 = -93.395479040704487
(gdb) p s% solver_dx(i_lnd,k)
$2 = -67.602505267054823
```

This is the first place where the density becomes very wrong. And in fact, $\ln\rho = -93.39 \approx \log\rho = -40.56$. We see that at this point, `p s% solver_dx(i_lnd,k)` = `-67.60...`, which is a really extreme correction.

Why is this?

To figure this out, we need to dive deeper into the code responsible for the corrections.

```
> grep -R "s%.*solver_dx.*=" star/private
star/private/star_solver.f90:     s% solver_dx(i,k) = dxsave(i,k)
star/private/star_solver.f90:     s% solver_dx(i,k) = s% solver_dx(i,k) + s% x_scale(i,k)*soln(i,k)
star/private/star_solver.f90:     s% solver_dx(i,k) = s% solver_dx(i,k) + coeff*s% x_scale(i,k)*soln(i,k)
star/private/star_solver.f90:     s% solver_dx(i,k) = dxsave(i,k) + s% x_scale(i,k)*soln(i,k)
star/private/star_solver.f90:     s% solver_dx(i,k) = dxsave(i,k) + coeff*s% x_scale(i,k)*soln(i,k)
star/private/star_solver.f90:     s% solver_dx(i_var,k+k_off) = save_dx(i_var,k+k_off) + delta_x
star/private/star_solver.f90:     s% solver_dx(i_var_sink,k+k_off) = save_dx(i_var_sink,k+k_off) - delta_x
star/private/star_solver.f90:     s% solver_dx(i_var,k+k_off) = save_dx(i_var,k+k_off)
star/private/star_solver.f90:     s% solver_dx(i_var_sink,k+k_off) = save_dx(i_var_sink,k+k_off)
star/private/alloc.f90:           s% solver_dx(1:nvar,1:nz) => s% solver_dx1(1:nvar*nz)
star/private/alloc.f90:           s% solver_dx(1:nvar,1:nz) => s% solver_dx1(1:nvar*nz)
star/private/struct_burn_mix.f90: s% solver_dx(j1,k) = s% xh(j1,k) - s% xh_start(j1,k)
star/private/struct_burn_mix.f90: s% solver_dx(j1,k) = s% xa_sub_xa_start(j2,k)
```

I think the easiest way to proceed is to actually *watch* `s% solver_dx(i,k)`. In particular, we want to watch $i = 1$ and $k = 1$, where k is the zone, and $i = 1$ corresponds to the $\ln\rho$.

To set this up, we need to make sure we can access the right object, so we have to take some preparation steps. This is the full list of commands.

```
> OMP_NUM_THREADS=1 /usr/bin/gdb ./star
(gdb) b do_solver
(gdb) run
(gdb) c
Continuing.

Breakpoint 1.2, star_solver::do_solver (s=0xe7c0e0
 ↪  <__star_data_def_MOD_star_handles>, nvar=13,
 ↪  af1=<error reading variable: value requires 23821592
 ↪  bytes, which is more than max-value-size>,
    ldaf=39, neq=76141,
     ↪  skip_global_corr_coeff_limit=.TRUE.,
     ↪  gold_tolerances_level=1,
     ↪  tol_max_correction=0.0030000000000000001,
     ↪  tol_correction_norm=3.0000000000000001e-05,
    work=<error reading variable: value requires 61521928
     ↪  bytes, which is more than max-value-size>,
     ↪  lwork=7690241,
    iwork=<error reading variable: value requires 304564
     ↪  bytes, which is more than max-value-size>,
     ↪  liwork=76141, convergence_failure=.TRUE., ierr=0)
     ↪  at ../private/star_solver.f90:109
109               subroutine do_solver( &
(gdb) delete 1
(gdb) set $star = s
(gdb) watch $star% solver_dx(1,1)
(gdb) b star_solver.f90:848
```

Okay! Let's look at the results:

```
Hardware watchpoint 2: $star% solver_dx(1,1)

Old value = 0
New value = 0.012397313114644006
apply_coeff (just_use_dx=<optimized out>, coeff=1,
 ↪  soln=<error reading variable: value requires 609128
 ↪  bytes, which is more than max-value-size>,
    dxsave=<error reading variable: value requires 609128
     ↪  bytes, which is more than max-value-size>,
     ↪  nz=5857, nvar=<optimized out>) at
     ↪  ../private/star_solver.f90:996
996                         do i=1,nvar
(gdb) c
Continuing.

Breakpoint 3, adjust_correction (_err_msg=256,
 ↪  _err_msg=256, ierr=0, err_msg=..., coeff=1, slope=0,
 ↪  f=0,
    grad_f=<error reading variable: value requires 609128
     ↪  bytes, which is more than max-value-size>,
     ↪  max_corr_coeff=1, min_corr_coeff_in=<optimized
     ↪  out>) at ../private/star_solver.f90:848
848                         call do_equations(ierr)
(gdb) p iter
$1 = 1
(gdb) c
Continuing.
     1  1  coeff 1.0000 avg resid  0.691E-05     max
      ↪  resid dv_dt     1  0.83380E+00 mix type xx000
      ↪  avg corr  0.206E-03      max corr lnd  1986
      ↪  0.53235E-02 mix type 00000  avg+max corr+resid

Hardware watchpoint 2: $star% solver_dx(1,1)

Old value = 0.012397313114644006
New value = 0.33770784939148324
apply_coeff (just_use_dx=<optimized out>, coeff=1,
 ↪  soln=<error reading variable: value requires 609128
 ↪  bytes, which is more than max-value-size>,
    dxsave=<error reading variable: value requires 609128
     ↪  bytes, which is more than max-value-size>,
     ↪  nz=5857, nvar=<optimized out>) at
     ↪  ../private/star_solver.f90:996
996                         do i=1,nvar
(gdb) c
Continuing.

Breakpoint 3, adjust_correction (_err_msg=256,
 ↪  _err_msg=256, ierr=0, err_msg=..., coeff=1, slope=0,
 ↪  f=0,
    grad_f=<error reading variable: value requires 609128
     ↪  bytes, which is more than max-value-size>,
     ↪  max_corr_coeff=1, min_corr_coeff_in=<optimized
     ↪  out>) at ../private/star_solver.f90:848
848                         call do_equations(ierr)
(gdb) p iter
$2 = 1
(gdb) c
Continuing.
     1  2  coeff 1.0000 avg resid  0.683E-05     max
      ↪  resid dv_dt     1  0.82426E+00 mix type xx000
      ↪  avg corr  0.573E-04      max corr lnd     2
      ↪  0.12612E-01 mix type x0000  avg+max corr+resid

Hardware watchpoint 2: $star% solver_dx(1,1)

Old value = 0.33770784939148324
New value = -67.602505267054823
apply_coeff (just_use_dx=<optimized out>, coeff=1,
 ↪  soln=<error reading variable: value requires 609128
 ↪  bytes, which is more than max-value-size>,
    dxsave=<error reading variable: value requires 609128
     ↪  bytes, which is more than max-value-size>,
     ↪  nz=5857, nvar=<optimized out>) at
     ↪  ../private/star_solver.f90:996
996                         do i=1,nvar
(gdb) c
Continuing.

Breakpoint 3, adjust_correction (_err_msg=256,
 ↪  _err_msg=256, ierr=0, err_msg=..., coeff=1, slope=0,
 ↪  f=0,
    grad_f=<error reading variable: value requires 609128
     ↪  bytes, which is more than max-value-size>,
     ↪  max_corr_coeff=1, min_corr_coeff_in=<optimized
     ↪  out>) at ../private/star_solver.f90:848
848                         call do_equations(ierr)
(gdb) p iter
$3 = 1
(gdb)
```

Clearly, we are getting a wrong `s% solver_dx(1,1)` from `apply_coeff`. It gets changed like this:

```
s% solver_dx(i,k) = s% solver_dx(i,k) + s%
 ↪  x_scale(i,k)*soln(i,k)
```

I inspected `s% x_scale` and `soln` throughout the Newton Iterations. The scale is consistent, while the solution at some point just becomes very large.

It also makes sense that we see the following in the terminal directly following the wrong density:

```
do_eos_for_cell: get_eos ierr           -1
do_eos returned ierr           -1
set_hydro_vars failed in call to set_micro_vars
                              set_hydro_vars returned ierr
                               ↪  -1
         eval_equations: set_solver_vars returned ierr
          ↪  -1
```

This is because directly following the call to `apply_coeff`, the routine `adjust_correction` calls `do_equations`, which calls `eval_equations` > `set_solver_vals` > `set_vars_for_solver` > `set_hydro_vars` > `set_micro_vars` > `set_eos` > `do_eos` > `foreach_cell` > `do_eos_for_cell` > `get_eos`.

This is the place where the first error occurs due to a density that is too small.

References

Karakas, A. I. & Lugaro, M. (2016) *Stellar Yields from Metal-rich Asymptotic Giant Branch Models*. ApJ825, 26.